

Assignment – 2

Choosing Architecture for a Scalable Food Delivery Platform

Case Study: A Swiggy / Zomato-style On-Demand Delivery System

Name	Dhuneshwaran V
Reg. no	192324169
Course Name	Software Engineering
Course Code	CSA1001

1. Problem Context

A food-delivery startup building a Swiggy/Zomato-style platform must handle thousands of orders per minute during lunch and dinner peak windows, while staying fast and available during off-peak hours when traffic is a fraction of that load. The system connects three very different actors in real time - customers placing orders, restaurants accepting and preparing them, and delivery partners picking up and dropping off food - each with its own latency, reliability, and scaling needs. The CTO's core question is which foundational architecture style will let the platform scale elastically, stay resilient when one part of the system misbehaves, and support fast, independent feature delivery as the product grows.

This report evaluates three candidate architecture styles - monolithic, microservices, and serverless - against the platform's real-time, high-throughput, multi-actor requirements, recommends a concrete architecture, explains the role event-driven design plays in it, and proposes a starting set of microservices.

2. Architecture Options Compared

Before recommending a direction, it helps to lay out how each candidate style behaves under the platform's specific pressures: bursty peak-hour load, independent team ownership of features (ordering, payments, logistics), and the need for parts of the system to fail without taking the whole platform down.

Criteria	Monolithic	Microservices	Serverless (FaaS)
Scaling	Whole app scales together; wasteful and slow at peak	Each service scales independently to match its own demand	Auto-scales per function, but cold starts hurt latency-sensitive paths
Fault isolation	Single point of failure - one bug can crash everything	A failing service is isolated; rest of platform keeps running	Isolated by function, but stateful long-running flows are awkward
Team velocity	Deploys are coupled; teams block each other	Teams own and deploy services independently	Fast for small functions, but orchestration across many gets complex
Real-time / stateful workflows	Simple to reason about, but does not scale well	Well-suited with an event bus coordinating long-running order/delivery flows	Weaker fit - execution time limits complicate order tracking
Operational cost at scale	Cheap initially, expensive to over-provision at peak	Pay for what each service needs; more DevOps investment required	Cheap at low/bursty volume, can spike unpredictably at sustained high volume

3. Recommended Architecture

Recommendation: A microservices architecture, composed around business capabilities (orders, payments, delivery, restaurants, notifications) and coordinated through an event-driven backbone (a message broker such as Kafka or RabbitMQ), fronted by an API Gateway. Latency-tolerant, bursty background jobs (e.g. generating daily restaurant reports, sending marketing notifications) can be offloaded to serverless functions, but the core order-processing path should live in dedicated, always-on microservices.

3.1 Three Technical Reasons for This Choice

- **Independent, workload-matched scaling:** Order volume, payment volume, and delivery-assignment volume do not spike together or by the same factor, so each service can be scaled independently - for example, running many more Order Service and Delivery Service instances at lunch rush without over-provisioning the Restaurant Service or Notification Service.
- **Fault isolation and graceful degradation:** With services split by capability, a spike or bug in one area - say, a slow third-party payment gateway - degrades only that service instead of bringing down order placement or delivery tracking platform-wide, which matters when thousands of orders are in flight every minute.
- **Independent deployability and technology fit:** Separate services let independent teams build, test, and deploy the Orders, Payments, and Delivery domains on independent release cycles and even in different languages/runtimes, shortening the feedback loop for new features (e.g. dynamic pricing, live rider tracking) without risking a platform-wide regression.

A pure monolith cannot meet the independent-scaling and fault-isolation needs above once order volume grows past a few thousand orders per minute, and a fully serverless design struggles with the long-lived, stateful workflow a food order represents (placed, accepted, preparing, picked up, delivered). Microservices is therefore the better primary architecture, with serverless used selectively for bursty, stateless side-tasks.

3.2 High-Level System Diagram

The diagram below shows client apps talking to the platform through a single API Gateway, which routes requests to independent microservices. Services communicate state changes through a shared event bus rather than calling each other directly, and each service owns its own data store.

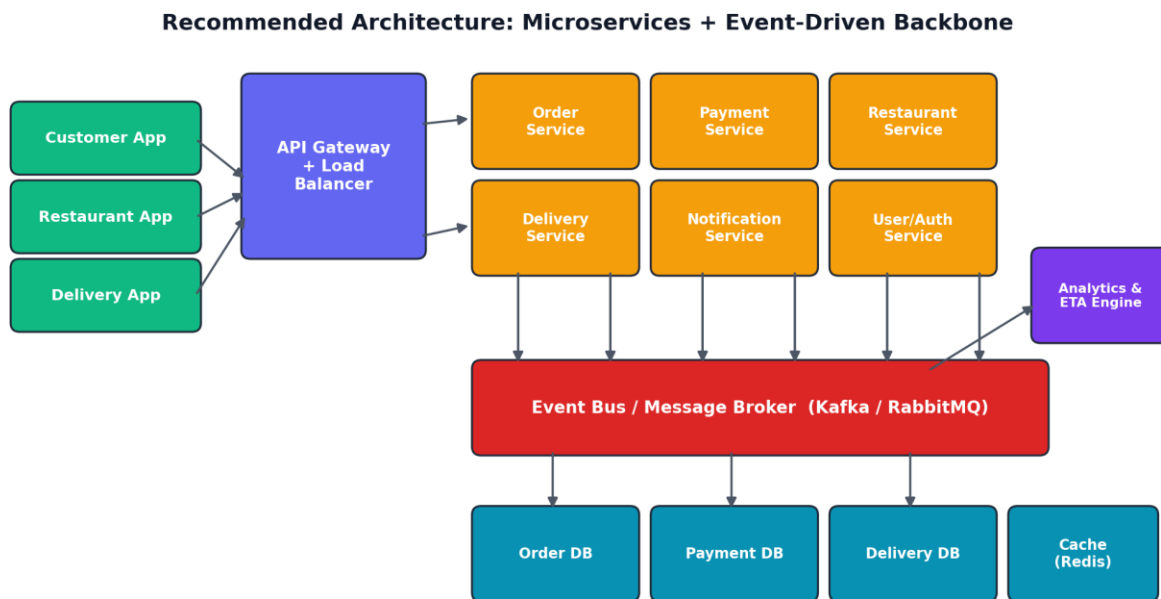
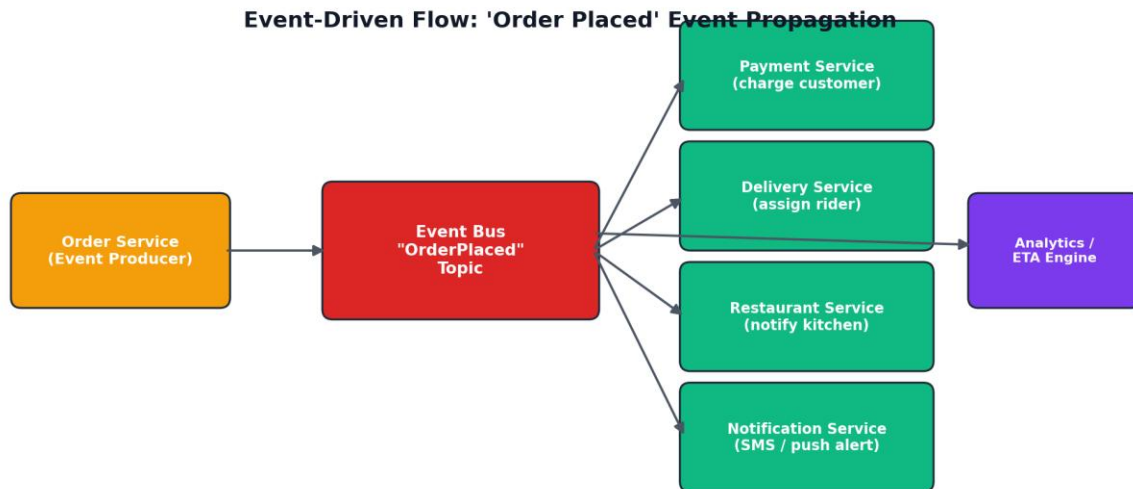


Figure 1: Recommended microservices architecture with an event-driven backbone.

4. Role of Event-Driven Architecture

In a microservices platform, services need to react to what other services do - a payment must be confirmed before a kitchen starts cooking, and a rider must be assigned once food is ready - without those services calling each other directly and waiting on a response. Event-driven architecture (EDA) solves this by having services publish events (facts about something that happened, such as "OrderPlaced" or "PaymentConfirmed") to a shared event bus, and having any interested service subscribe and react on its own schedule.



Each consumer reacts independently and asynchronously - no direct service-to-service calls, no blocking waits.

Figure 2: A single OrderPlaced event fanning out to independent consumer services.

4.1 Key Benefits for This Platform

- **Loose coupling and non-blocking flow:** The Order Service publishes an event and moves on immediately; it never blocks waiting for Payment, Delivery, or Notification services to finish their work, which keeps order placement fast even when a downstream service is slow.
- **Natural load buffering at peak traffic:** During a lunch-hour surge, the message broker absorbs the burst of events and lets each consumer service drain the queue at its own sustainable pace, instead of every request hitting every service synchronously at the same instant.
- **Resilience to partial failures:** If the Notification Service is temporarily down, its events simply wait in the queue and are processed once it recovers - order placement, payment, and delivery assignment are unaffected.
- **Easy extensibility for new features:** New consumers - such as a fraud-detection service or a real-time analytics dashboard - can subscribe to existing events (e.g. OrderPlaced, PaymentConfirmed) without any change to the services that publish them.
- **Built-in audit trail:** Every event is a durable, timestamped record, so the full lifecycle of an order (placed, paid, accepted, picked up, delivered) can be reconstructed for auditing, debugging, or analytics.

5. Proposed Microservices

The table below lists a starting set of microservices, the business capability each one owns, and the key events it publishes or consumes on the event bus.

Microservice	Responsibility	Key Events Published / Consumed
Order Service	Creates and tracks orders; owns the order state machine from cart to delivery	Publishes: OrderPlaced, OrderCancelled. Consumes: PaymentConfirmed, DeliveryCompleted
Payment Service	Processes payments, refunds, and settlement with payment gateways	Consumes: OrderPlaced. Publishes: PaymentConfirmed, PaymentFailed
Delivery Service	Matches orders to available riders, tracks live location and ETA	Consumes: OrderReadyForPickup. Publishes: RiderAssigned, DeliveryCompleted
Restaurant Service	Manages restaurant menus, availability, and order acceptance/preparation status	Consumes: PaymentConfirmed. Publishes: OrderAccepted, OrderReadyForPickup
Notification Service	Sends SMS, push, and email updates to customers, restaurants, and riders	Consumes: OrderPlaced, RiderAssigned, DeliveryCompleted
User / Auth Service	Handles registration, login, profiles, and access tokens for all three apps	Publishes: UserRegistered. Consumed by most services for identity checks

At minimum, Orders, Payments, and Delivery form the core trio: Orders drives the workflow, Payments gates progression into fulfilment, and Delivery closes the loop with the customer. Restaurant, Notification, and User/Auth services round out a production-ready first release.

6. Scaling and Resilience Strategy for Peak Hours

Choosing microservices with an event-driven backbone is only half the story - the platform also needs concrete mechanisms to survive lunch and dinner rushes without falling over. The following practices apply the architecture in Section 3 to the thousands-of-orders-per-minute requirement.

- **Auto-scaling per service:** Each microservice runs behind a horizontal pod/instance autoscaler tied to CPU, memory, or queue-depth metrics, so the Order and Delivery services can scale out to many more instances during rush and scale back in afterwards to control cost.
- **Caching hot data:** Frequently read, rarely changing data - restaurant menus, delivery zones, promotional pricing - is served from an in-memory cache (e.g. Redis) so the primary databases are not hit on every request during peak load.
- **Database sharding for high-write tables:** Large tables such as orders and payments are partitioned by region or customer ID across multiple database nodes, keeping any single node's write load manageable as order volume grows.
- **Rate limiting at the gateway:** The API Gateway enforces per-user and per-IP rate limits and can apply request throttling or queuing during extreme spikes, protecting backend services from being overwhelmed by retries or abusive traffic.
- **Circuit breakers between services:** If the Delivery Service's rider-matching logic slows down, an internal circuit breaker stops cascading failures into the Order Service by failing fast and falling back to a simplified matching strategy instead of queuing requests indefinitely.

- **Consumer-side parallelism on the event bus:** Consumer groups on the event bus (e.g. multiple Notification Service instances reading the same topic) let message processing scale horizontally, so a backlog of events built up during a spike drains quickly once extra capacity comes online.

7. Conclusion

For a food-delivery platform that must absorb thousands of orders per minute at peak while staying responsive off-peak, a microservices architecture built around an event-driven backbone gives the best combination of independent scaling, fault isolation, and team velocity. Event-driven communication keeps services loosely coupled and non-blocking, lets the platform absorb traffic bursts gracefully, and provides a natural audit trail of every order's lifecycle. A monolith cannot scale its parts independently, and a fully serverless approach struggles with the platform's long-lived, stateful order workflow - making microservices, with serverless reserved for bursty background jobs, the recommended path forward.